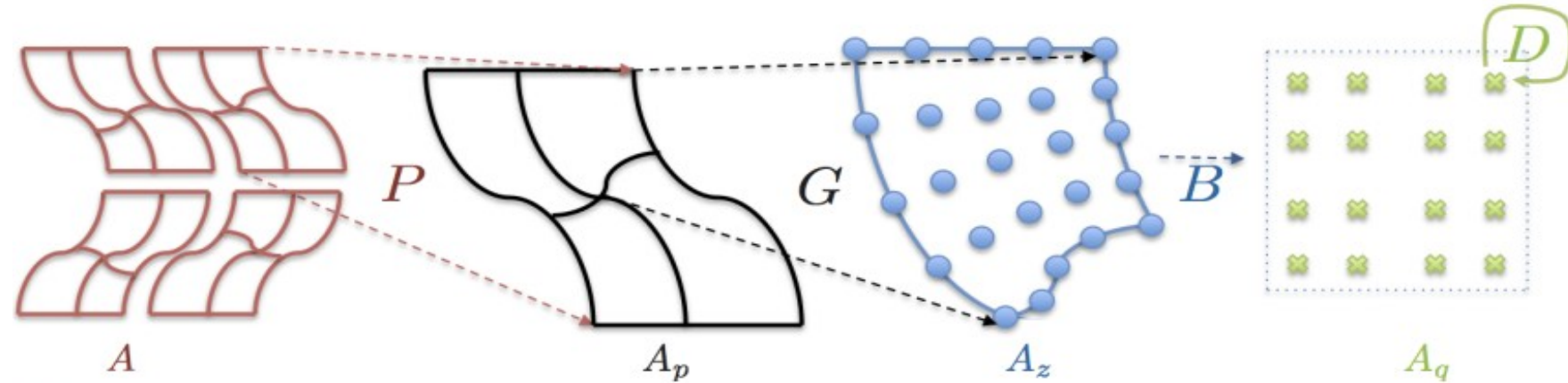


Low-level API (libCEED)

$$A = P^T G^T B^T D B G P$$



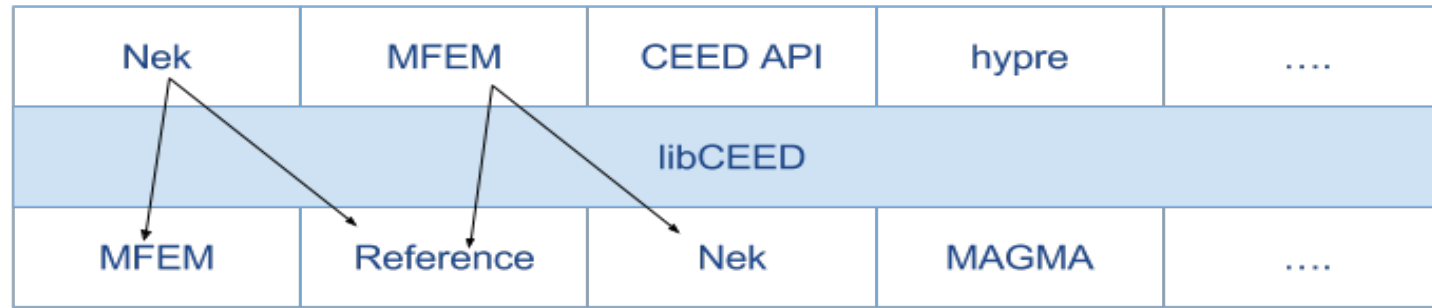
Parallel, *Mesh Topology*, *Finite Element Basis*,
Geometry/Physics

- A suite of higher order operator discretization that enables efficient evaluation using BP winners as back-end evaluation kernels.
- For each operator we have:
 - Data structure representing the assembled operator
 - Operator assembly routine
 - Operator action routine
- Building blocks for the high level API:
 - Multiple element types, e.g. quad+tri
 - Variable polynomial orders
- Use *efficient* operator representations

```
void diffusion_action
(
    double *y,           /* result, input-output vector */
    double *x,           /* input vector */
    double *D,           /* nqpt_1d x nqpt_1d x nelelem */
    double *B1d,         /* nqpt_1d x ndof_1d dense matrix */
    double *B1d_t,       /* transpose of B1d */
    double *G1d,         /* nqpt_1d x ndof_1d dense matrix */
    double *G1d_t,       /* transpose of G1d */
    int *dof_offsets,    /* array of size
                        (ndofs_1d ^ dim) x nelelem
                        representing a boolean P */

    int ndof_1d,         /* number of 1D dofs (points) */
    int nqpt_1d,         /* number of 1D quadrature points */
    int nelelem,         /* number of elements */
    int ndim,            /* Dimension of the problem */
    int backend,         /* Backend type */
    int vtype,           /* Vector type */
    int olayout,         /* Operator type */
);
```

Low-level API (libCEED)



Current Status

- Reference, Nek5000 and MFEM backend implementations (More optimizations to be done).
- MFEM can call Reference and Nek5000 backends for Mass and Diffusion actions.
- Working on using MFEM and Reference backends from Nek5000

Future Plans

- Figure out the correct API that should be exposed to the high-level user applications.
- Implement more actions, combined actions, etc.
- Expand and optimize for various devices, GPUs, KNLs, etc.